Instituto Superior de Formación Técnica Nº 151
Carrera: Analista en Sistemas.
Alumna: LUZURIAGA, Silvana.
3 Año. Algoritmos y Estructuras de Datos III.

Marco Teórico:
Parte I – Introducción a la Materia.

1. Explicar la cita docente
   "***Estudiaremos las estructuras de datos para aprender a escribir programas más eficientes***"

   La razón detrás de esto es que nuestras ambiciones crecen a la par de nuestras capacidades computacionales, la revolución en el poder de computo simplemente aumenta la necesidad de ser eficientes a medida que intentamos realizar tareas mas complejas. La creación de programas eficientes no se trata de "trucos" de programación, sino que se basa fundamentalmente en una buena organización de la información y en la selección de buenos algoritmos.

2. Explicar los 2 enfoques presentados que deben cumplir el diseño de algoritmos.
   * Diseñar un algoritmo que sea fácil de entender, codificar y depurar, lo cual es una preocupación principal de la Ingenieria de Software.
   * Diseñar un algoritmo que haga un uso eficiente de los recursos de la computadora.

3. Que se entiende por estructuras de datos
   *En el sentido general una estructura de datos es cualquier representación de datos y sus operaciones asociadas.
   * Mas comúnmente se refiere a una organización o estructura diseñada para una colección de datos y elementos.
   * Fisicamente, hace referencia a una colección de elementos y a la manera en que esta se almacena en la memoria de la computadora o en algún dispositivo de memoria secundaria, lo cual determina como se pueden recuperar esos datos.

4. Porque debemos medir la eficiencia de los Algoritmos?
   * Se debe medir la eficiencia porque existen muchas alternativas para solucionar un problema, y necesitamos poder seleccionar el algoritmo mas eficiente que consuma menor recursos.
   * Solo a través de dicha medición podemos determinar que estructura de datos o algoritmo de nuestra caja de herramientas es el mas apropiado para un problema nuevo.
   * Esta eficiencia se mide habitualmente utilizando el Analisis Asintotico para estimar el costo de tiempo y memoria.

5. Porque hablamos de una “Filosofía” de las Estructuras de Datos?
   * Porque los informáticos de hoy deben tener un conocimiento profundo de los principios detrás del diseño eficiente de programas, ya que las experiencias de la vida ordinaria a menudo no aplican a la hora de diseñar software.
   * La gente escribe programas para resolver problemas específicos, y es fundamental analizar primero el problema para determinar los objetivos de rendimiento y seleccionar la estructura adecuada.
   * Ignorar este análisis y aplicar representaciones inapropiadas suele dar como resultado programas lentos.

6. Por que relacionamos las Estructuras de Datos con los TDA?
   * Un Tipo de Dato Abstracto (TDA o ADT) es la realización lógica de un tipo de datos, definido por un conjunto de valores y sus operaciones, ocultando los detalles de implementación (encapsulación).
   * Una estructura de datos, en cambio, es la implementación física de ese TDA.
   * Juntos (el TDA y su implementación) forman una clase en lenguajes orientados a objetos y nos permiten gestionar la enorme complejidad del software a través de la abstracción.

7. Profundizar la Frase: “El costo de la Solución es la Cantidad de Recursos que la Solución Consume”
   - Una solución se considera eficiente si requiere menos recursos que las demás alternativas conocidas.
   - Al evaluar un algoritmo, este coste se cuantifica usando la “Complejidad Temporal” (el tiempo de computo necesario para ejecutar el programa) y la “Complejidad Espacial” (la cantidad de memoria que el programa utiliza para almacenar sus variables).

8. Cuales “Deberían” ser los Pasos para seleccionar una Estructura de Datos?
   - Analizar el problema para determinar cuales son las operaciones básicas que deben ser soportadas (por ejemplo: insertar, eliminar, o buscar un elemento)
   - Cuantificar las limitaciones de recursos para cada una de esas operaciones.
   - Seleccionar la estructura de datos que mejor se adapte a estos requisitos analizados.

9. Relacionar estructuras de Datos Vs. Costos y Beneficios?
   - Esta relación se explica con el concepto de “trade-offs” o equilibrios.
   - Existen costos y beneficios asociados a la cantidad de espacio y tiempo que necesita cada estructura de datos para realizar sus operaciones típicas.
   - Al no existir soluciones netamente mejores ni recetas universales, el programador debe elegir la opción mas adecuada dependiendo del contexto del problema.

10. Describir por que las Estructuras de Datos tienen una estructura “Lógica” y una “Física”, argumentar.
    - Tienen una forma lógica que esta definida por el Tipo de Datos Abstracto (TDA), especificando que representa el tipo y que operaciones se pueden hacer con él.
    - Tienen una forma física, que es la estructura de datos real que implementa ese TDA en la memoria de la computadora.
    - Esta distinción es importante porque, por ejemplo, el concepto lógico de “Lista” puede

implementarse físicamente de varias formas distintas, como a través de arreglos o mediante llistas enlazadas dinámicas.

11. Relacionar las Estructuras de Datos con Patrones.
    * Existen diferentes niveles de abstracción sobre los cuales clasificamos las soluciones en la programación, recibiendo el nombre de patrones.
    * Mientras que los patrones arquitectónicos definen la estructura básica y los patrones de diseño refinan el diseño detallado, las estructuras de datos entran en juego en los modismos (idioms) y en la implementación física dentro de esos marcos para resolver problemas recurrentes específicos.

12. Armar una Relación Narrativa que incluya: Problema, Algoritmo y Programa.

    Un problema es una tarea que al realizar puede verse matemáticamente como una función o un mapeo entre salidas y entradas, sin incluir restricciones de como resolverlo. Para solucionar ese problema, se requiere de un algoritmo, que es una receta, proceso o serie de pasos concretos y no ambiguos que transforman esa entrada en la salida esperada. Finalmente, ese algoritmo cobra vida a través de un programa, que es una representación concreta o instanciación de dicho algoritmo escrito en un lenguaje de programación particular.